

Trabalho Prático 1

Um trabalho prático entregue à Disciplina de Algoritmos II, DCC, UFMG

Projeto: Comida di Buteco 2026 - Belo Horizonte

Isaac Reyes e João Araldi

Link Repositório: https://github.com/jparaldi/ktree_dibuteco26

1. Introdução

O projeto surge da necessidade de otimizar a localização de estabelecimentos gastronômicos participantes do evento "Comida di Buteco 2026" em Belo Horizonte. Dada a vasta distribuição geográfica dos bares pela capital mineira, o desafio técnico reside em realizar buscas espaciais que sejam rápidas e precisas, partindo de um endereço fornecido pelo usuário.

Tradicionalmente, uma busca linear $O(n)$ em uma lista de coordenadas seria ineficiente para grandes volumes de dados. Por isso, a implementação de uma KD-Tree, que é uma estrutura de particionamento de espaço que permite reduzir drasticamente o número de comparações necessárias para encontrar pontos vizinhos, é viável.

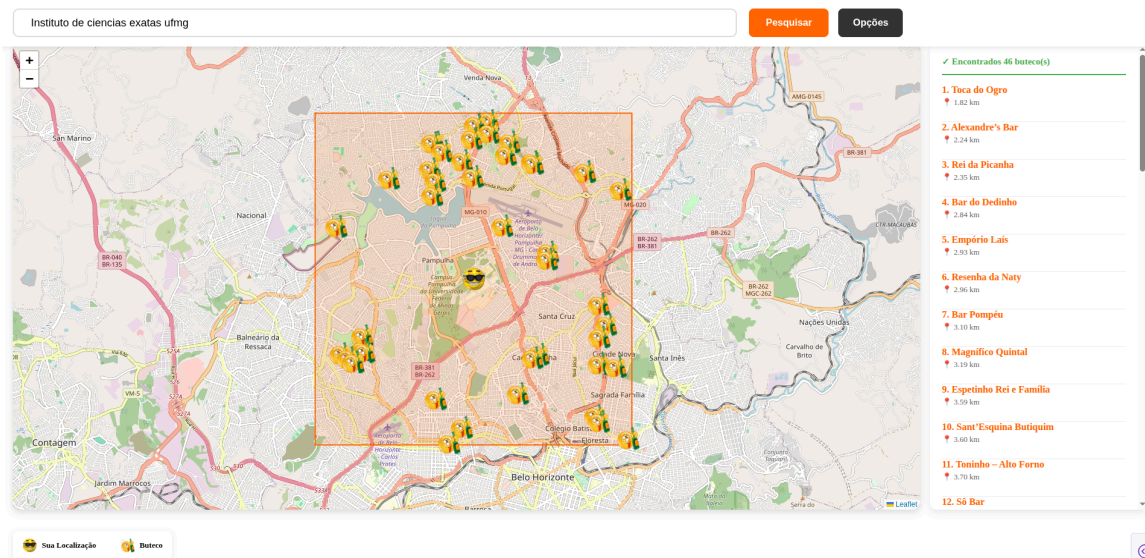


Figura 1: visão da interface web

2. Objetivos

- Implementar uma estrutura de dados KD-Tree bidimensional.
- Realizar buscas por intervalo (*range search*) retangulares e circulares.
- Criar interface web interativa para visualização com Dash e Leaflet.
- Ordenar resultados por distância (no trabalho, usamos *haversine*).

3. Descrição técnica da solução

3.1. KD-Tree

A KD-Tree implementada no arquivo `src/core/kdtree.py` organiza os bares em uma árvore binária onde cada nível alterna a divisão entre latitude e longitude. A construção segue o princípio da mediana, garantindo que a árvore permaneça balanceada com complexidade de construção $O(n \cdot \log(n))$.

A eficiência da KD-Tree reside na capacidade de "podar" ramos inteiros da árvore que não possuem interseção com a área de busca durante os dois modos de busca. Na busca retangular, define-se um bounding box [min, max] para latitude e longitude, já na circular usa-se um raio em km e a distância de haversine para filtrar pontos.

*Nota sobre a implementação da "Diagonal": A orientação do trabalho mencionava uma "diagonal de busca", mas o conceito era vago em termos de geometria. Decidimos interpretar como um ****quadrado centrado no ponto de busca****, onde a diagonal corresponde ao lado do quadrado (ou aproximadamente num ângulo de 45°). Dessa forma, o parâmetro "Diagonal de Busca (km)" define a meia-distância em cada direção (latitude e longitude) a partir do ponto central, criando uma busca simétrica e intuitiva para o usuário.*

3.2. Ordenação por distância

Em `src/core/geometry.py`, implementamos duas métricas de distância:

- **Haversine:** Calcula a distância real na superfície terrestre, considerando a curvatura da Terra. É a métrica utilizada na busca circular e para a ordenação final dos resultados.
- **Euclidiana:** Calcula a distância simples em graus, sendo mais rápida, mas menos precisa para grandes distâncias.

Inicialmente, houve incerteza sobre qual métrica usar. A Euclidiana é computacionalmente mais rápida e suficiente para pequenas distâncias, enquanto a Haversine é mais precisa, porém mais pesada. Mas, no final, optamos por utilizar a Haversine em ambos os contextos (busca e ordenação) por considerar a experiência do usuário e a precisão geográfica como prioridades. Usuários esperam a distância real em linha reta na superfície terrestre.

3.3. Interface web

A interface web foi desenvolvida com o objetivo de atender ao requisito de um sistema interativo que integre, de forma transparente, a entrada de dados, a configuração da busca e a visualização dos resultados.

A aplicação é estruturada em três módulos principais:

- `app.py`, responsável pela inicialização;
- `layout.py`, que define a organização dos componentes visuais;
- `callbacks.py`, onde está concentrada a lógica reativa.

Tal estrutura permite a separação de responsabilidades e melhora a legibilidade do código, uma vez que cada arquivo contém uma parte das regras de negócio, sem acoplamento ou dependências circulares.

O mapa interativo atua como principal elemento de visualização, exibindo tanto o ponto de referência quanto os estabelecimentos encontrados dentro da região definida. Esses pontos são atualizados à medida que os parâmetros de busca são modificados. Complementarmente, um painel de resultados apresenta os bares filtrados, incluindo nome, endereço e distância em relação ao ponto informado, já ordenados de forma crescente, garantindo aderência aos requisitos do trabalho .

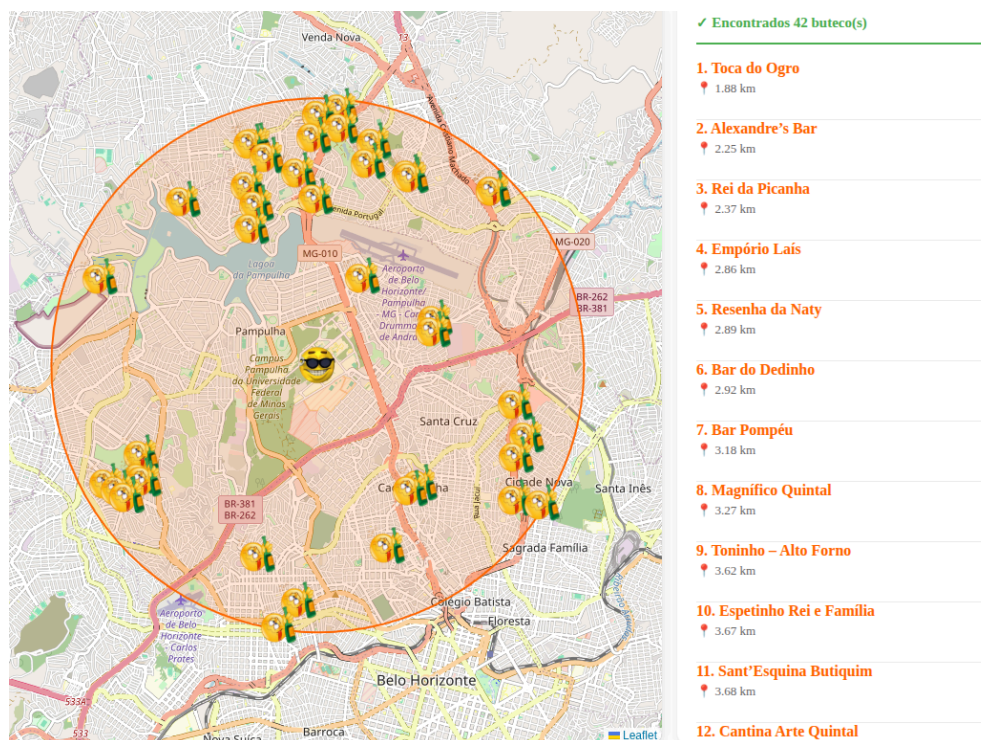


Figura 2: resultado de uma consulta do sistema

Cabe destacar uma limitação relacionada ao pré-processamento dos dados: dos 127 butecos presentes na base original, 12 não foram incluídos na aplicação final devido a falhas no processo de geocodificação. Esses casos correspondem a endereços que não puderam ser convertidos de forma confiável em coordenadas geográficas, seja por inconsistências na escrita, ausência de informações suficientes ou limitações da própria API utilizada. Como a estrutura de busca depende diretamente dessas coordenadas, tais registros foram descartados para evitar comportamentos incorretos.

A interação começa pela barra de entrada, onde o usuário fornece um endereço que é convertido em coordenadas geográficas e utilizado como ponto central da busca. A partir desse ponto, um conjunto de parâmetros pode ser ajustado por meio de um modal de opções, incluindo o tamanho da região de interesse, a quantidade máxima de resultados retornados e o tipo de busca. Esses controles permitem ao usuário alterar dinamicamente a área considerada, refletindo diretamente na projeção da região no mapa e nos resultados obtidos, conforme especificado no enunciado .

Opções de Busca

Quantidade de Resultados:

1 10 20 50 100 127 100

Diagonal de Busca (km):

0.3km 5 10km 20km 30km 35km 5

☐ Busca circular

Reset OK

Figura 3: Controles de busca e filtragem do sistema

A comunicação entre os componentes é realizada por meio de callbacks, que implementam um modelo reativo: qualquer alteração relevante, seja no endereço ou nos parâmetro, aciona automaticamente a recomputação da busca e a atualização sincronizada do mapa e da lista, sem necessidade de recarregamento da página. Esse comportamento reforça a integração entre a estrutura de dados implementada e a interface, permitindo uma exploração contínua e interativa do espaço de busca.

Por fim, aspectos visuais como a customização dos marcadores foram tratados de maneira descontraída. Essas decisões não impactam a lógica da aplicação nem a execução dos algoritmos.



Figuras 4 e 5: elementos visuais decorativos usados no sistema

3.4. Geocoding

O processo de geocodificação converte os endereços dos bares em coordenadas geográficas, sendo essencial para a construção da KD-Tree e para as buscas espaciais. Essa etapa foi implementada em `geocoding.py` utilizando o serviço Nominatim do OpenStreetMap.

A fim de evitar chamadas repetidas à API e reduzir custo computacional, o geocoding dos bares foi realizado previamente uma única vez, com os resultados armazenados na pasta `data/processed`. Durante a execução da aplicação, apenas novos endereços inseridos pelo usuário passam por esse processo. Caso algum endereço não seja encontrado, o sistema trata a falha retornando uma mensagem de erro, evitando o uso de dados inválidos.

4. Exemplos práticos

A fim de testar a corretude e o bom funcionamento da busca na árvore, foram realizadas várias consultas em endereços diversos e os resultados foram comparados com a base de dados.

Para fins de demonstração, foram separadas duas consultas que ilustram bem os dois tipos de busca.

4.1. Busca retangular simples

- **Entrada:** Praça da Liberdade, Belo Horizonte.
- **Configuração:** resultados limitados a 5, diagonal de busca de 2 km e tipo de busca retangular.

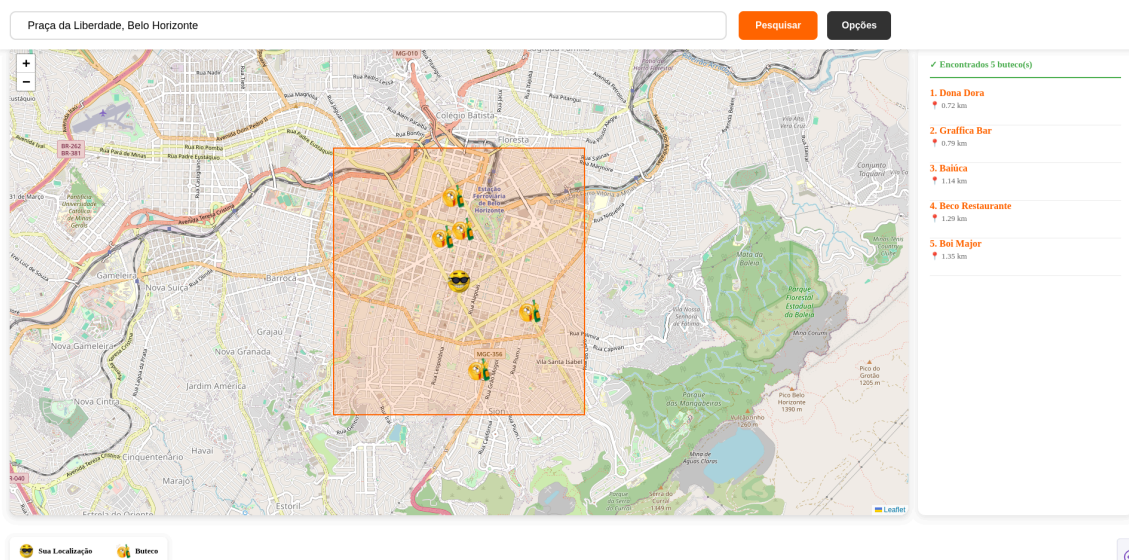


Figura 6: Resultados da busca retangular simples

4.2. Busca circular com raio maior

- **Entrada:** Av. Getúlio Vargas, Belo Horizonte.
- **Configuração:** resultados limitados a 10, diagonal de busca de 5 km e tipo de busca circular.

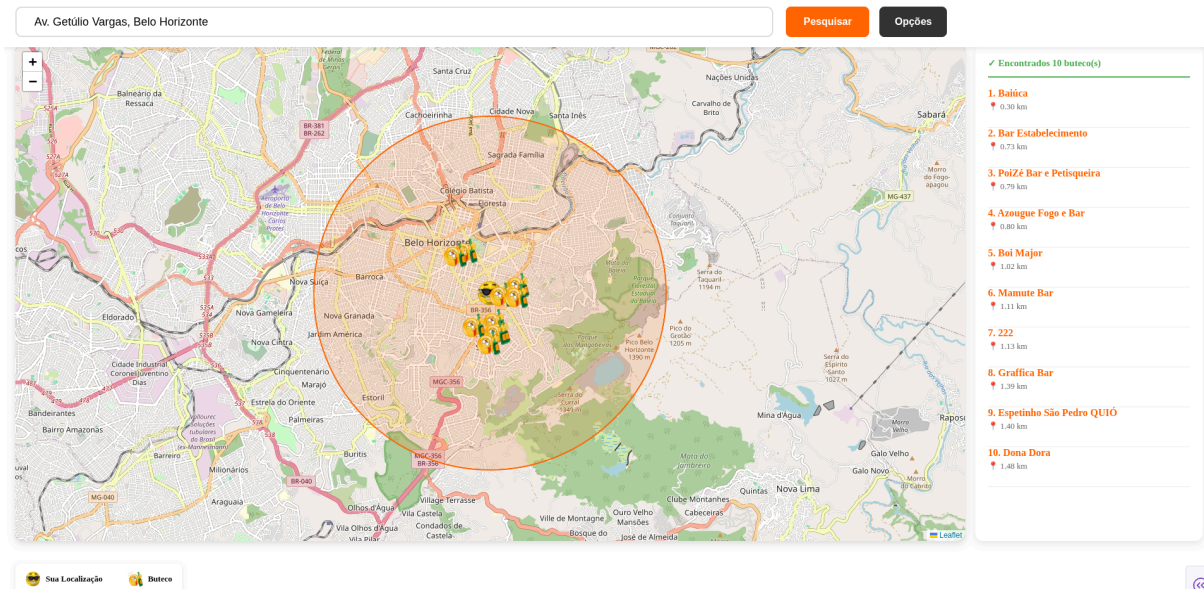


Figura 7: resultados da busca circular com raio maior

5. Comentários adicionais

O sistema foi estruturado de forma modular, com separação clara entre camadas como aplicação, núcleo algorítmico, serviços e utilidades, o que contribui para legibilidade, manutenção e evolução do código. A organização das pastas reflete diretamente essa divisão de responsabilidades, isolando componentes como a KD-Tree, os algoritmos de busca, o processamento geométrico e a interface interativa.

Do ponto de vista algorítmico, a construção da KD-Tree ocorre em $O(n \cdot \log(n))$, enquanto as buscas apresentam complexidade média $O(\log(n) + k)$, onde k é o número de pontos retornados. Além disso, decisões e auxílios estão documentados no código por meio de comentários.

Em termos de experiência do usuário, a aplicação incorpora elementos interativos e feedback visual que tornam a exploração dos dados mais intuitiva.

Em relação à robustez, foram implementados diversos tratamentos de erro para garantir estabilidade mesmo diante de entradas inválidas ou inconsistências nos dados. Endereços que não podem ser geocodificados são tratados com mensagens de erro amigáveis, evitando falhas na execução; registros com coordenadas nulas no CSV são ignorados durante o carregamento; e buscas sem resultados retornam um aviso claro ao usuário.

Sobre o desempenho, mesmo para áreas maiores (como 20 km), a KD-Tree reduz o custo de busca ao podar ramos irrelevantes, enquanto o cálculo de distância (Haversine) ocorre em tempo constante $O(1)$ por ponto avaliado.

Por fim, possíveis falhas externas, como problemas no carregamento do mapa, podem ser diagnosticadas a partir da verificação dos serviços do OpenStreetMap, isolando a origem do erro sem comprometer o restante da aplicação.

6. Busca Circular

Conforme sugerido nas especificações do trabalho, foi implementada a busca circular (*Hyperspherical regions*) como uma funcionalidade adicional para complementar a busca ortogonal. Diferente da busca retangular, que avalia as interseções através de *bounding boxes*, a busca circular filtra os pontos garantindo que a distância real entre o endereço central e os bares seja menor ou igual ao raio estipulado.

Para manter a eficiência da estrutura e evitar uma varredura linear $O(n)$, a busca na KD-Tree foi adaptada para podar ramos de forma inteligente. O algoritmo verifica a distância mínima possível entre a região delimitada por um nó da árvore e o ponto central; se essa distância mínima for maior que o raio de busca, toda a subárvore é descartada, removendo comparações inúteis.

7. Tratamento de casos de borda

Durante o desenvolvimento e testes, alguns cenários extremos foram mapeados e tratados:

- **Raio sem correspondências:** Quando o usuário define um raio muito restrito em uma região sem bares participantes, a KD-Tree finaliza a busca rapidamente e retorna uma lista vazia. A interface foi programada para capturar esse estado e exibir um aviso claro ao usuário, evitando erros na renderização do mapa ou da tabela de resultados.
- **Limites de arredondamento e precisão:** Como utilizamos a distância Haversine em ambos os contextos de busca e ordenação, pontos situados exatamente no limite do raio geográfico não sofrem com distorções de escala de projeção (comuns no cálculo de distâncias Euclidianas simples). Isso garante precisão para o usuário final independentemente da curvatura da Terra.